

Graphs

LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about graphs and the types
- Represent a graph
- Perform graph traversal
- Know the AND/OR graph
- Know the bi-connected component
- Understand topological sorting

One disadvantage of the tree data structure is that it is completely hierarchical in that it represents only parent–child relationships and only indirectly a sibling relationship. When we lift this limitation, we get the data structure graph. Generally, no restriction is imposed on the number of vertices nor on the number of connections a vertex can have with other vertices. We shall define the graph more precisely before describing several applications of graphs.

A simple graph $G = (V, E)$ consists of a non-empty set of vertices V and possibly an empty set of edges, each edge being a set of two vertices from V .

8.1 INTRODUCTION

A graph can be defined (and represented) as a set of vertices and a set of edges. An edge is a connection between two vertices. The graph shown in Figure 8.1 is a set of vertices

$$V = \{1, 2, 3, 4, 5, 6\}$$

and a set of edges

$$E = \{ (1, 2), (1, 5), (2, 3), (3, 4), (4, 5), (4, 6) \}$$

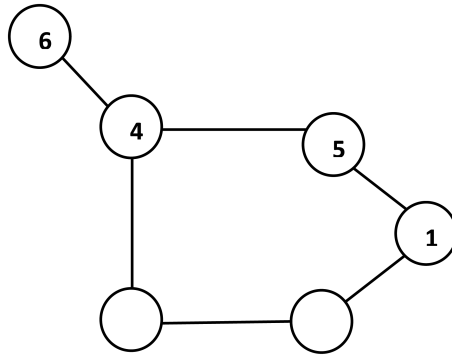


Figure 8.1 A graph

- *Undirected graphs:* In an undirected graph, the order of vertices in the pairs representing the edge set does not matter (as in the graph in Figure 8.1). In undirected graphs, the edges are drawn as lines between pairs of vertices. The adjacency relation in an undirected graph is symmetric. So, if $u \sim v$ is a relation representing the fact that (u, v) is an edge, then $(v \sim u)$ also holds.
- *Directed graphs:* In a directed graph, the order of vertices in the representation is important. If $u \sim v$ is true (representing the fact that u to v is an edge), $v \sim u$ may not be true, which means that v to u may not be an edge. We usually use arrows to represent edges in a directed graph. An arrow is drawn from u to v only if (u, v) is in the edge set. A simple path in a graph from u to v is defined as a set of vertices $\{u, v_1, v_2, \dots, v_n, v\}$ such that the edges $(u, v_1), (v_1, v_2), \dots, (v_n, v)$ are all in the edge set and $v_1, v_2, \dots, v_n$ are all distinct.

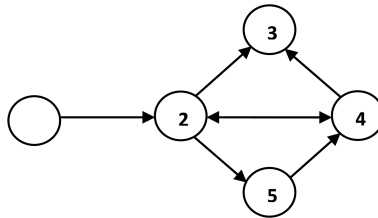


Figure 8.2 Directed graph

For the graph shown in Figure 8.2, the vertex set is $\{1, 2, 3, 4, 5\}$ and the edge set is $\{(1, 2), (2, 3), (2, 4), (2, 5), (4, 2), (4, 3), (5, 4)\}$.

- *Vertex-labeled graph:* Each vertex in addition to the label it carries also contains some data. In the graph in Figure 8.3, the labels to the vertices are shown outside the nodes and the data is shown inside the nodes.

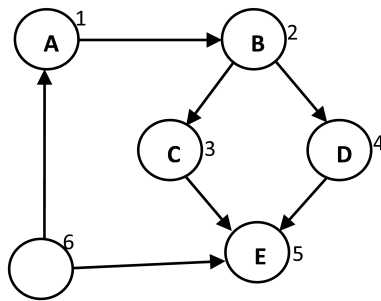


Figure 8.3 Vertex-labeled graph

We can show the data in the vertex set, like

$$V = \{(1, A), (2, B), (3, C), (4, D), (5, E), (6, F)\}$$

and the edge set as

$$E = \{(1, 2), (2, 3), (2, 4), (3, 5), (4, 5), (6, 1), (6, 5)\}$$

- *Cyclic graph*: A cyclic graph is a directed graph with at least one cycle. A cycle in a graph is a directed path from any vertex to itself. In Figure 8.4, there are two cycles and they are

$$2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 6 \rightarrow 2$$

and

$$4 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 2 \rightarrow 4$$

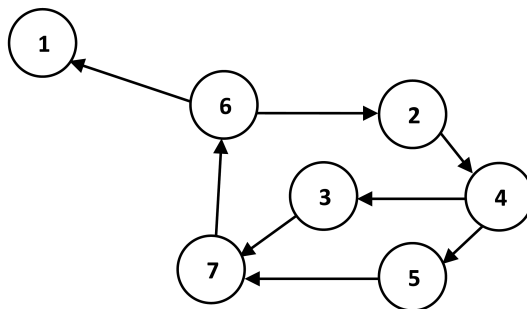


Figure 8.4 Cyclic graph

- *Edge-labeled graph*: In an edge-labeled graph, the edges are associated with labels. The labels are shown as a triple (u, v, X) showing an edge from u to v with label X .

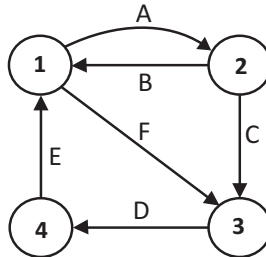


Figure 8.5 Edge-labeled graph

In the graph in Figure 8.5, the vertex set is $\{1, 2, 3, 4\}$ and the edge set is $\{(1, 2, A), (2, 1, B), (2, 3, C), (3, 4, D), (4, 1, E), (1, 3, F)\}$, where

A is the label associated with $(1, 2)$.

B is the label associated with $(2, 1)$.

C is the label associated with $(2, 3)$.

D is the label associated with $(3, 4)$.

E is the label associated with $(4, 1)$.

F is the label associated with $(1, 3)$.

- *Weighted graphs*: If the labels associated with the edges are numbers indicating the importance of the edge, we call the graph a weighted graph (Figure 8.6). The numbers can be distances between two cities where the nodes represent cities or the cost associated with the completion of phase I of a project and taking it to phase II, where the nodes represent different phases of a project.

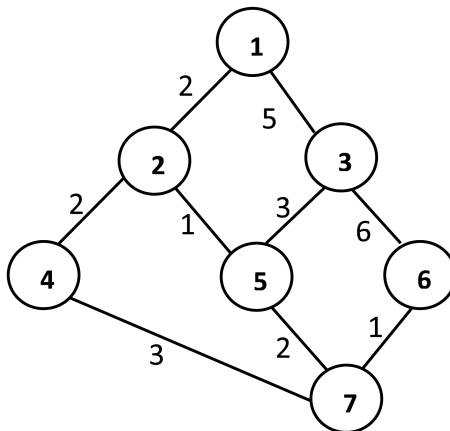


Figure 8.6 Weighted graph

- *Directed acyclic graphs (DAGs)*: A DAG is a directed graph with no cycles.

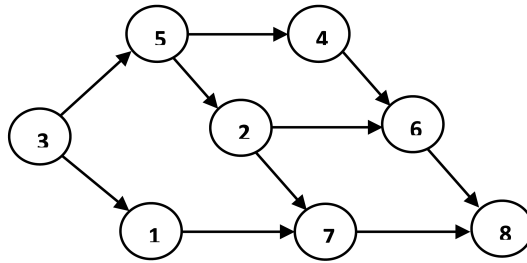


Figure 8.7 Directed acyclic graph

The vertex set is $\{1, 2, 3, 4, 5, 6, 7, 8\}$.

The edge set is $\{(1, 7), (2, 6), (3, 1), (3, 5), (4, 6), (5, 2), (5, 4), (6, 8), (7, 2), (7, 8)\}$.

The graph shown in Figure 8.7 is a directed acyclic graph since it contains no cycles. Vertices with only in-arrows are called sinks. In Figure 8.7, vertex 8 is a sink.

- *Adjacent vertex*: Graphs are often used to represent physical entities (a network of roads, relationships between people, etc.) inside a computer. A good choice of representation depends on the operations that the program needs to perform on the graph to solve a problem.

In a graph, two vertices are called adjacent if there is an edge connecting them. The degree of a vertex in a graph is the number of edges associated with it.

Consider the graph in Figure 8.8. It is a weighted undirected graph.

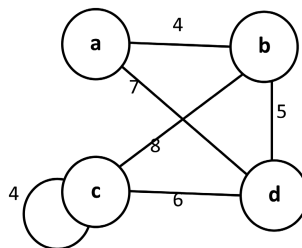


Figure 8.8 Adjacent vertex

- The degree of a is 2.
 - The degree of b is 3.
 - The degree of c is 4.
 - The degree of d is 3.
- The self-loop counts for two edges.

Theorem: Let G be an undirected graph with V vertices and N edges, then

$$2N = \sum_{v \in V} \text{degree}(v)$$

Proof: If $u \sim v$ is an edge, each of the vertices is of degree 1. An edge therefore contributes 2 to the total degree of the graph. Hence, if there are N edges, their contribution to the total degree is $2N$.

In a directed graph, we define the in-degree of a vertex as the number of edges directed toward it and the out-degree as the number of edges directed away from it (Figure 8.9).

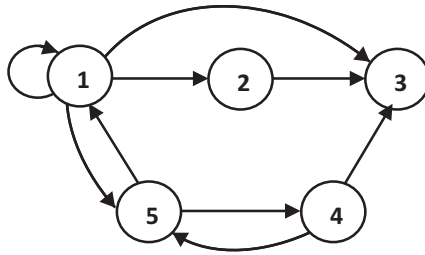


Figure 8.9 Directed graph

- The in-degree of 1 is 2
- The out-degree of 1 is 3.
- The in-degree of 2 is 1.
- The out-degree of 2 is 1.
- The in-degree of 3 is 3.
- The out-degree of 3 is 0.
- The in-degree of 4 is 1.
- The out-degree of 4 is 2.
- The in-degree of 5 is 2.
- The out-degree of 5 is 2.

Theorem: Let G be a directed graph (or a multi-graph) with V vertices and N edges. Then

$$2N = \sum_{v \in V} \text{in-degree}(v) + \sum_{v \in V} \text{out-degree}(v)$$

This is just an extension of the earlier theorem. The in-degree counts the edges into a node and the out-degree counts the edges going out of a node and the sum, therefore, gives the total number of edges, counted twice, i.e., $2N$.

- **Complete graph:** The graph in which all the nodes have the same degree is called a complete graph.

A complete graph, with n vertices, denoted as K_n , is a simple graph in which there is an edge between every pair of vertices (Figure 8.10).

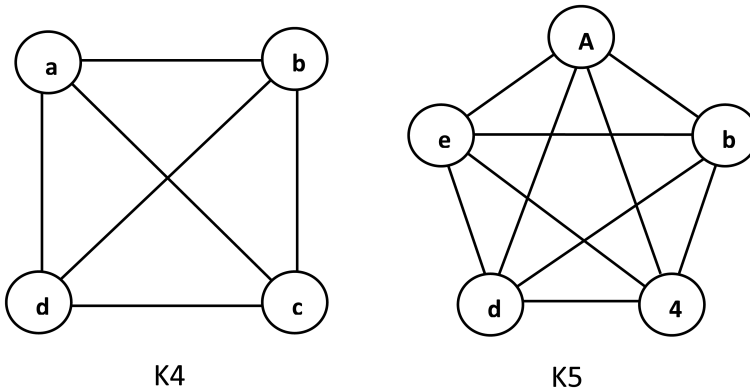


Figure 8.10 Complete graph

- **Connectivity:** The path in a graph is a sequence of distinct vertices connected by edges. Vertex v is reachable from u , if there is a path from u to v . A graph is connected if there is a path between every pair of vertices.

Example

Given a graph with seven vertices, three of them of degree 2 and four of them of degree 1, is the graph connected?

Answer: No

Reason: The number of edges is equal to $[(3 \times 2) + (4 \times 1)]/2 = 5$. The minimum number of edges, if all of them are connected, is six.

If every pair of vertices is connected in a directed graph, then it is said to be strongly connected.

A directed graph is weakly connected if the underlying undirected graph is connected.

Theorem: In an undirected graph, there are at most $n(n-1)$ edges.

Proof: We shall prove the theorem by induction. If $V = 1$, there are no edges. Hence the theorem is true.

Let $V = n$. Then the number of edges is equal to $n(n-1)/2$ (hypothesis).

Let $V = n+1$. The number of edges added is maximum n , when the new vertex is connected to all other n vertices. Therefore, the maximum number of edges when $V = n + 1$ is $n(n-1)/2 + n = (n+1)n/2$.

8.2 GRAPH REPRESENTATION

There are two ways to represent a graph:

- Adjacency list
- Adjacency matrix

8.2.1 Adjacency list

Adjacency list representation is used when the number of edges is less than $V \times \log V$. The adjacency list representation of a graph G consists of an array of linked lists, one for each vertex. Consider the graph in Figure 8.11.

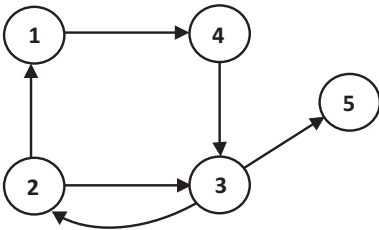


Figure 8.11 Directed graph

The corresponding representation is in Figure 8.12.

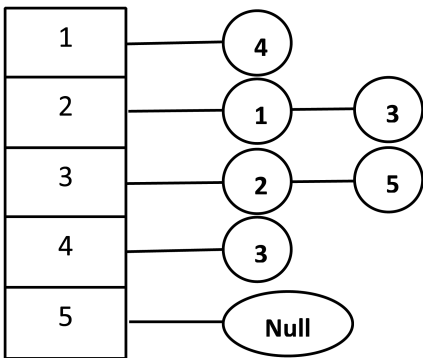


Figure 8.12 Adjacency list

Vertices in adjacency lists are stored in an arbitrary order. A potential disadvantage of adjacency list representation is there is no easy way to find if there is an edge between two vertices.

8.2.2 Adjacency matrix

If M is the matrix representing a graph G , then

$$M_{ij} = \begin{cases} 1 & \text{if there is an edge between vertices } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

	1	2	3	4	5
1	0	0	0	1	0
2	1	0	1	0	0
3	0	1	0	0	1
4	0	0	1	0	0
5	0	0	0	0	0

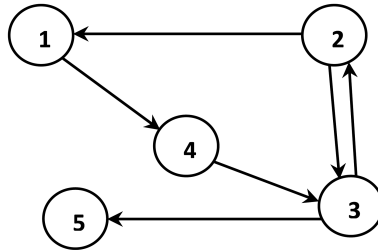


Figure 8.13 Adjacency matrix

The graph shown in Figure 8.12 can be represented by the matrix in Figure 8.13.

A tree can be considered as a connected graph with no cycles.

A connected graph with V vertices and $V - 1$ edges must be a tree.

A weighted graph is also represented as an adjacency matrix where, if there is an edge, the entry is the weight and not 1. The adjacency matrix of the weighted graph is shown in Figure 8.14.

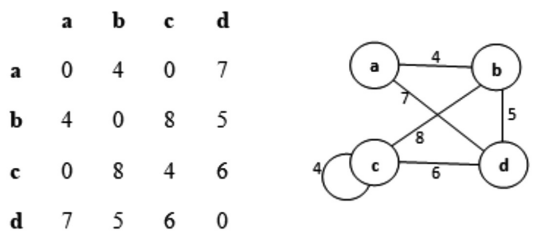


Figure 8.14 Adjacency matrix for a weighted graph

8.3 TRAVERSALS

In tree traversals, we start at the root and follow the edges until we encounter a null node. A node is never visited twice since there are no cycles. However, in graph traversals it is possible we visit the same node many times by following the edges. Hence it is necessary to keep track of nodes already visited and ignore them the second time. This may be accomplished by maintaining a Boolean array “visited,” which may be of the size of the number of nodes, one entry for each node of the graph, and is initialized to false. This array keeps track of the nodes already visited so far and ignores them when we reach them again.

8.3.1 Depth-first search

The depth-first search (DFS) visits all child vertices, recursively, before visiting the sibling vertices. In other words, it traverses the depth of any particular path before exploring its breadth. Often a stack is used for implementing the algorithm. There is nothing called a “root” vertex in a graph. We arbitrarily choose a vertex and start the search from there. It then iteratively visits adjacent unvisited vertices until it can no longer find an unvisited vertex from its present position. In other words, all adjacent vertices of the current position are already visited. The algorithm, then, “backtracks” along the previously visited vertices until it finds a vertex that is not yet visited. It follows the new path backtracking whenever necessary (if it finds dead ends) and ends only when the algorithm backtracks to the root.

Algorithm 1

- 1.Algorithm dfs(G,v)
- 2.Mark v visited and push it onto a stack
- 3.While stack is not empty do

4. Pop a vertex from the stack
5. Call it u
6. Mark it visited, if not already marked visited.
7. Push the adjacent vertices of u onto the stack, after marking them visited.
8. Go to step 4
9. End while

Consider the graph in Figure 8.15.

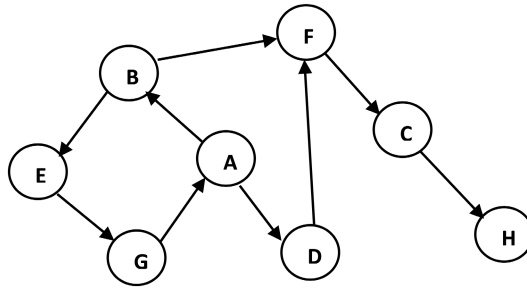


Figure 8.15 Directed graph

Let us start from A. Mark it *visited*. Push it onto the stack (Figure 8.16a). *Visited* is a Boolean array of the size of the number of nodes; one entry for each node. True if the node is visited, false otherwise.

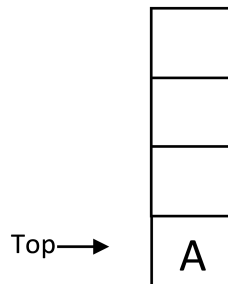
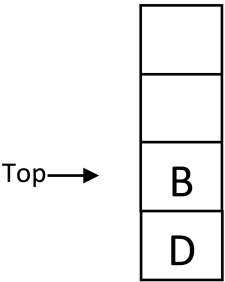


Figure 8.16a A is in the stack

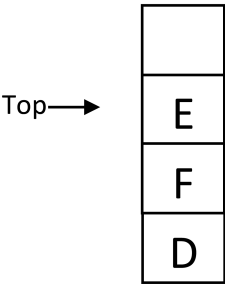
Pop an element from the stack, show it in the output, and push its adjacent vertices D, B in that order onto the stack (any of the adjacent sides can be pushed onto the stack, but we push them in alphabetical order) after marking them visited. The stack position now is as in Figure 8.16b.



Output: A

Figure 8.16b A is popped and D, B are in the stack

We have B on top of the stack, pop it out and show it in the output. Its adjacent nodes are A, E, F. A is already visited. Push F and E onto the stack, in that order, after marking them visited. The stack position now is as in Figure 8.16c.



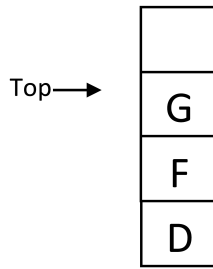
Output: A B

Figure 8.16c A and B are popped and D, F, E are in the stack

The top element E in the stack is popped out and shown in the output. Its adjacent is G. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16d.

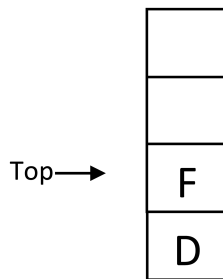
We have G on top of the stack. Pop it out and show it in the output. As there is no element to push, the position of the stack is as in Figure 8.16e.

The top element F in the stack is popped out and shown in the output. Its adjacent is C. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16f.



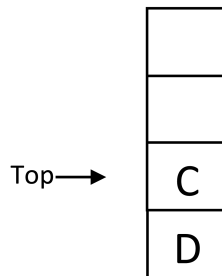
Output: A B E

Figure 8.16d A, B and E are popped and D, F, G are in the stack



Output: A B E G

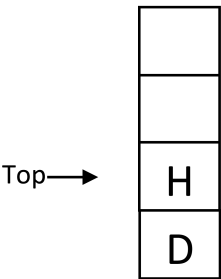
Figure 8.16e A, B, E and G are popped and D, F are in the stack



Output: A B E G F

Figure 8.16f A, B, E, G and F are popped and D, C are in the stack

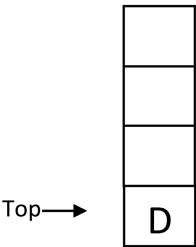
The top element C in the stack is popped out and shown in the output. Its adjacent is H. It is marked as visited and pushed onto the stack. Now the stack is as shown in Figure 8.16g.



Output: A B E G F C

Figure 8.16g A, B, E, G, F and C are popped and D, H are in the stack

We have H on top of the stack. Pop it out and show it in the output. As there is no element to push, the position of the stack is as in Figure 8.16h.



Output: A B E G F C H

Figure 8.16h A, B, E, G, F, C and H are popped and D is in the stack

We have D on top of the stack. Pop it out and show it in the output. As there is no element to push, the stack is empty.

The sequence of nodes contained in the output is

A B E G F C H D

Python program I

```
graph = {
    'E' : ['B', 'G'],
    'G' : ['A', 'E'],
    'B' : ['A', 'E', 'F'],
    'A' : ['B', 'D', 'G'],
    'D' : ['A', 'F'],
    'F' : ['B', 'C', 'D'],
    'C': ['F', 'H'],
    'H': ['C']
}

visited = [] # List for visited nodes.
queue = []   #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:               # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')      # function calling
```

Output

Following is the Depth-First Search

```
A
B
E
G
F
C
H
D
```

8.3.2 Breadth-first search

In a breadth-first search (BFS), we start at any node and visit all the adjacent nodes. We use the data structure queue to determine the next node to be visited. We also maintain a Boolean array *visited* to keep track of all

the nodes processed so far. To illustrate the procedure, let us use the same graph we used to illustrate the depth-first traversal (Figure 8.17). We use a queue instead of a stack.

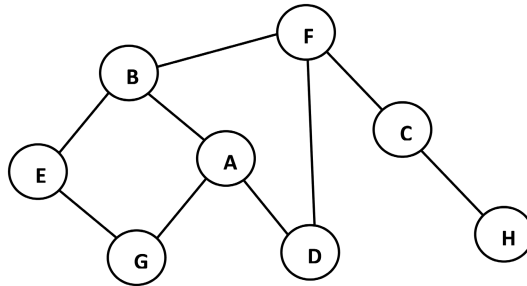


Figure 8.17 Undirected graph

We start from A. Mark it visited. Insert into the queue. Now remove A from the queue, display in the output and insert its adjacent vertices B D G in alphabetical order into the queue and mark them as visited.

The status of the queue now is shown in Figure 8.18a.

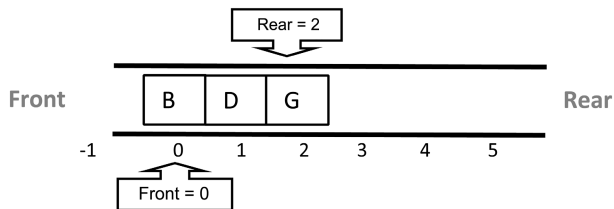


Figure 8.18a B D G are in the queue

Visited  A G D B

Output A

B is removed from the queue, and is put in the output. Its adjacent nodes E and F are inserted into the queue in that order and marked as visited. The current status of the queue and the array ‘visited’ are shown in Figure 8.18b.

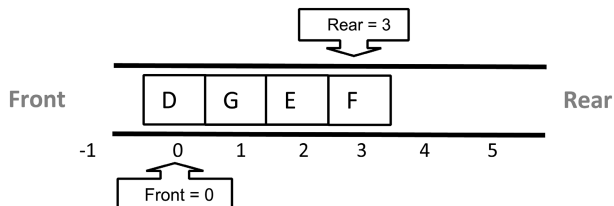


Figure 8.18b D G E and F are in the queue

Visited ☹️ A G D B E F
 Output ☹️ A B

The next vertex to be processed is D. It is removed from the queue and added to the output and its adjacent vertices A and F are already visited. The status of the queue at this position is shown in Figure 8.18c.

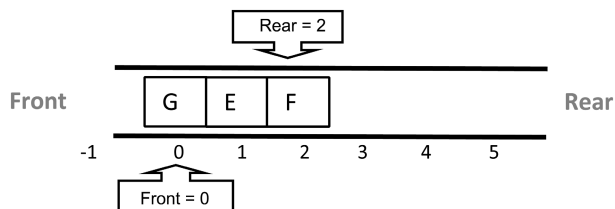


Figure 8.18c G E and F are in the queue

Visited ☹️ A G D B E F
 Output ☹️ A B D

The next vertex in the queue is G. It is removed from the queue and added to the output (Figure 8.18d).

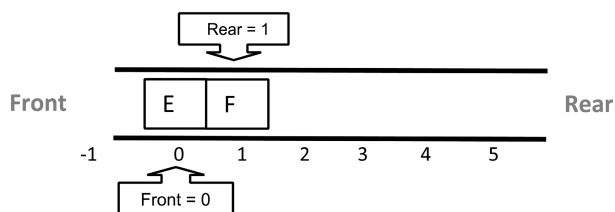


Figure 8.18d E and F are in the queue

Visited ☹️ A G D B E F
 Output ☹️ A B D G

G's neighbors are already visited. The next vertex E is taken out from the queue and added to the output (Figure 8.18e). The status of the queue, the array visited and output are

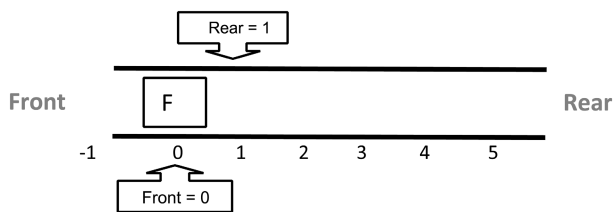


Figure 8.18e F is in the queue

Visited ☹ A G D B E
Output ☹ A B D G E F

Remove F from the queue and add to the output. Its neighbor C is added to the queue (Figure 8.18f).

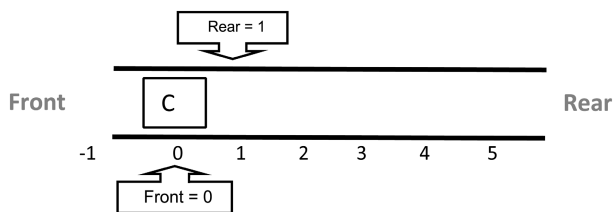


Figure 8.18f C is in the queue

Visited ☹ A B D G E F C
Output ☹ A B D G E F

Remove C from the queue and add to the output. Its neighbor H is added to the queue after marking it visited (Figure 8.18g).

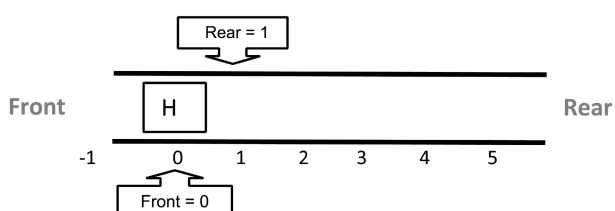


Figure 8.18g H is in the queue

Visited ☹ A B D G E F C H
Output ☹ A B D G E F C

Lastly, H has no unvisited neighbors; it is added to the visited and output. Now the queue is empty and the path of the breadth-first search is

A B D G E F C H

Python program 2

```
graph = {
    'E' : ['B','G'],
    'G' : ['A', 'E'],
    'B' : ['A','E','F'],
    'A' : ['B','D','G'],
    'D' : ['A','F'],
    'F' : ['B','C','D'],
    'C': ['F','H'],
    'H': ['C']
}

visited = [] # List for visited nodes.
queue = []    #Initialize a queue

def bfs(visited, graph, node): #function for BFS
    visited.append(node)
    queue.append(node)

    while queue:
        # Creating loop to visit each node
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

Output

Following is the Breadth-First Search
A B D G E F C H

8.4 AND/OR GRAPHS

AND/OR graphs are very useful for problem-solving. Suppose we want to solve problem A. Let us represent this as the root of a tree.

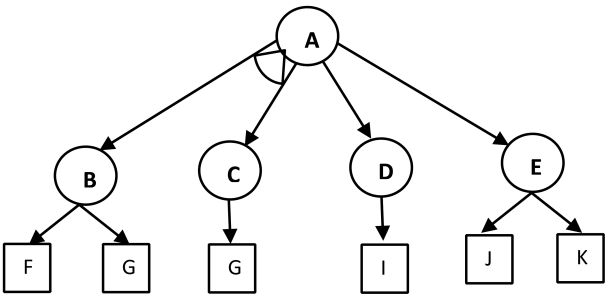


Figure 8.19a AND/OR graph

The tree structure in Figure 8.19a implies that, to solve problem A, solve both problems B and C or solve D or solve E. The tree structure can further be extended breaking down the problems B, C, D, E into sub-problems. The terminal square nodes represent solvable problems without further decomposition. As we can see, AND/OR graphs are not necessarily trees.

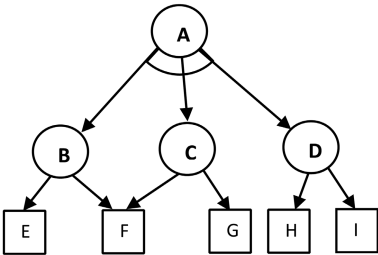


Figure 8.19b AND/OR graph

The diagram in Figure 8.19b is not a tree. Problem A can be solved by solving B, C, D. To solve B we have to solve E or F and the solution to F is also needed to solve C. Thus there is a cycle and hence is not a tree.

Let us assume that there is a cost associated with each edge. Therefore, the cost of the solution to a problem represented as an AND/OR graph is the sum of the edge costs. Consider the example in Figure 8.19c.

To solve P1 we have to solve P2 or P3 or P4. To solve P2 we have to solve P5 and P6. To solve P3 we have to solve both P6 and P7. Therefore, if we obtain the solution of P1 through P2, the cost is $1 + 1 + 2 = 4$. If we obtain the solution through P3, the cost is $3 + 2 + 1 = 6$. And through P4, it is 8. Therefore, the minimum cost path is $P1 \rightarrow P2 \rightarrow (P5 + P6)$.

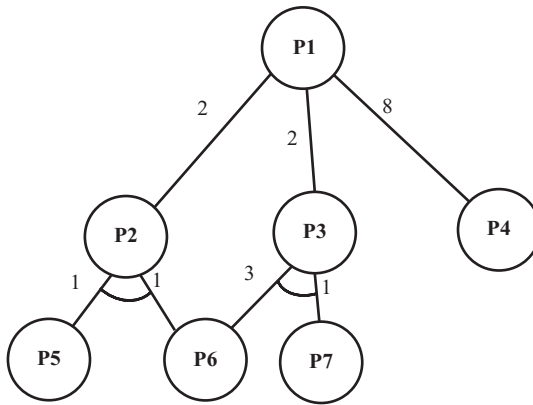


Figure 8.19c AND/OR graph

8.5 BI-CONNECTED COMPONENTS

Bi-connected components, also known as 2-connected components, are a type of maximal sub-graph in graph theory that maintains connectivity even after removing a vertex. They are connected at cut vertices, separating vertices or articulation points. These components are crucial in graph algorithms and network analysis, helping to identify critical points, analyze robustness, and identify bridges or articulation points affecting a graph's connectivity.

8.5.1 Connectivity

An undirected graph is said to be connected if there is a path between any two vertices. A graph is said to be n -connected if there are at least n different paths between any two vertices and the paths having no common vertices. A special type of graph is a 2-connected or bi-connected graph in which there are at least two different paths between any two vertices. A graph is not bi-connected if a vertex can be found that always has to be included in a path between at least two different vertices, say a and b . In other words, if this vertex is removed (along with incident edges), there is no way to reach from a to b and vice versa. This means that the graph is split into two or more sub-graphs. Such vertices are called articulation points or cut-vertices. If an edge causes its removal by a split of the graph into two sub-graphs, the edge is called a bridge. The sub-graphs obtained by removing the articulation points are called bi-connected components.

Consider the graph in Figure 8.20 (Horowitz, Sahni, Rajsekharan et al.).

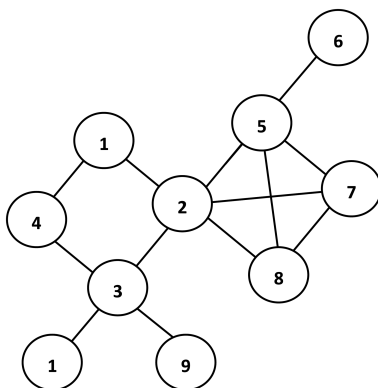


Figure 8.20 Bi-connected graph

Removal of vertex 2 creates two sub-graphs (Figure 8.21a).

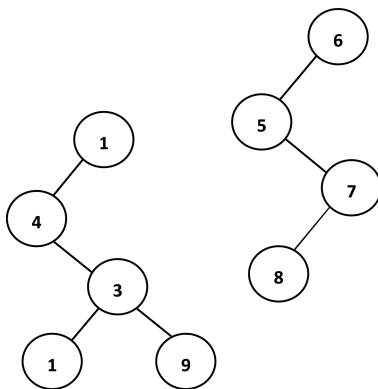


Figure 8.21a Bi-connected component

Hence vertex 2 is an articulation point. A graph is, thus, bi-connected if and only if it contains no articulation points. The sub-graphs obtained by removing the articulation points are called bi-connected components. In the case of a communication network, represented as a graph, it is not desirable to have any articulation points since any damage to the computer systems at the articulation points removes communication connection between several systems. Let $G = (V, E)$. We define a maximal bi-connected sub-graph of G as $G' = (V', E')$ if and only if there is no sub-graph $G'' = (V'', E'')$, which is bi-connected such that $V' \subset V''$ and $E' \subset E''$. A maximal bi-connected sub-graph is a bi-connected component.

Two bi-connected components can have at most one vertex in common and that point is an articulation point. Hence no edge can be in two bi-connected components. The bi-connected components of the previous graph are shown in Figure 8.21b.

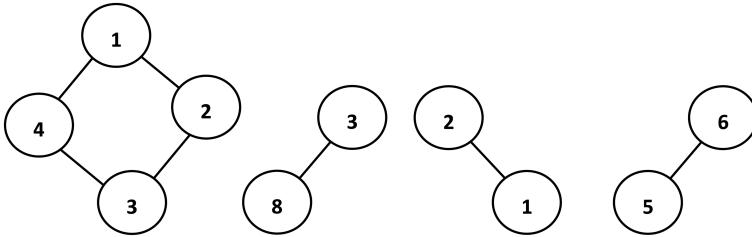


Figure 8.21b Bi-connected component

In order to construct a bi-connected graph, we have to make sure the articulation points are no longer articulation points by adding new edges.

Algorithm 2

```

1. Algorithm bi-connected(G)
2. for each articulation point a do
3. {
4.     let  $B_1, B_2, \dots, B_k$  be the bi-connected components
       containing the vertex a.
5. let  $v_i \neq a$  be a vertex in  $B_i$ ,  $1 \leq i \leq k$ 
6. add to G the edges  $\langle v_i, v_{i+1} \rangle$ 
7. }
```

This construction converts a graph G into a bi-connected graph G' . Now the problem that remains is to identify articulation points. Let us first identify the depth-first spanning tree of G , as shown in Figure 8.22.

The depth-first algorithm creates a tree with forward edges (edges included in the tree) and back edges (edges not included in the tree). Thus, a vertex v in this tree is an articulation point, if there is at least one sub-tree, whose vertices are not connected with any of v 's predecessors by a back edge because none of the predecessor vertices can be reached from the forward vertices. It is only through the back edge (which is not in the tree but is in the graph) that we can reach a predecessor. In Figure 8.22, the spanning tree is shown in double lines. The number outside the nodes, called the depth-first number (dfn), shows the order in which the nodes are visited. The edges shown in dotted lines are known as back edges, which means that they are in the original graph but not in the depth-first spanning tree.

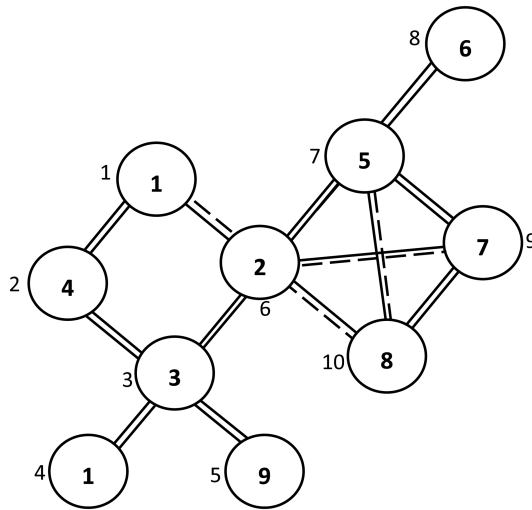


Figure 8.22 Bi-connected component

- The root node of the depth-first spanning tree is an articulation point if and only if it has two children. If we remove the root, the nodes in the left sub-tree cannot reach the nodes in the right sub-tree.
- The leaf node in the depth-first spanning tree is never an articulation point.
- Non-leaf, non-root node u is an articulation point if and only if no non-tree edge goes above u from a sub-tree below some child of u . This is because, if there is a non-tree edge connecting the children to the ancestors, the children remain connected to their ancestors, even when the parent is removed.

The presence of a back edge means the presence of an alternate path to the ancestors.

Let us define $L(u)$ as the lowest depth-first number that can be reached from u using a path of its descendants and a back edge. Applying the definition we have $L[1:10] = [1, 1, 1, 1, 6, 8, 6, 6, 5, 4]$.

Let us calculate some of these numbers.

$L[1] = 1$, since it can reach itself through $1 \rightarrow 4 \rightarrow 3 \rightarrow 2$ and the back edge $2 \rightarrow 1$.

$L[2] = 1$, since we can reach 1 from 2 through the back edge $2 \rightarrow 1$.

Similarly, $L[3] = 1$ and $L[4] = 1$.

$L[5] = 6$, since 5 can reach 2 through its descendants 7 and 8 and the back edge $8 \rightarrow 2$ and $\text{dfn}(2) = 6$.

$L[6] = \text{dfn}(6) = 8$, since it has no descendants.

$L[7] = 6$, since we can reach node 2 whose dfn is 6 through $7 \rightarrow 8$ and back edge $8 \rightarrow 2$. And similarly, $L[8] = 6$.

$L[9] = 5$ and $L[10] = 4$, their own depth-first numbers, since they have no descendants and no back edges.

If u is not a root, then u is an articulation point if and only if u has a child w such that $L(w) \geq \text{dfn}(u)$. Vertex 2 is an articulation point since its child 5 satisfies $L(5) \geq \text{dfn}(2)$, since $L(5) = 6$ and $\text{dfn}(2) = 6$.

Similarly, nodes 3 and 5 are articulation points.

8.6 SHORTEST-PATH PROBLEM

Let $G = (V, E)$ be a directed graph. The shortest path between two vertices in the graph is the path with shortest length (least number of edges) called the link distance. A breadth-first search discussed earlier is the algorithm for finding the shortest link distance from a single source vertex to all other vertices in the graph. A breadth-first search processes all vertices in the increasing order of distance from the source.

Now, let $G = (V, E)$ be a weighted graph. Let us use a weight function that maps the edges to real numbers:

$$w: E \rightarrow \mathbb{R}$$

if e is an edge, $w(e)$ is called the weight associated with the edge e . The length of the path from a vertex u to a vertex v is the sum of the weights associated with the edges that constitute the path between u and v . Let us denote the minimum distance between u and v as $d(u, v)$, if one exists.

8.6.1 Single-source shortest-path problem

Given a di-graph $G = (V, E)$ with non-negative edge-costs (weights) and a special vertex s , called the source vertex, determine the shortest path and distance between the source vertex and all other vertices.

Dijkstra's algorithm

1. Let $d(v)$ be the length of the shortest path from ' s ' to each vertex v .
2. Initialize $d(v)$ to ∞ for all vertices of the graph.
3. Let $d(s) = 0$.
4. Process vertices to find new paths and update $d(v)$.

The processing is done as follows:

- There is a shortest path from ' s ' to ' u ' with length $d(u)$.
- There is a path from ' s ' to ' v ' with length $d(v)$.
- Another path to ' v ' through ' u ', if it exists, has length $d(u) + w(u, v)$.
- If $d(u) + w(u, v) < d(v)$, then the old path is replaced by the new path and $d(v)$ is replaced by $d(u) + w(u, v)$.

The last statement can be made more precise by the following statement:

```

If( $d(u) + w(u,v) < d(v)$ )
{
     $d(v) = d(u) + w(u,v)$ ;
     $pred(v) = u$ ;
};

```

where $pred(v)$ means the predecessor of v .

Let us apply Dijkstra's algorithm to the di-graph in Figure 8.23.

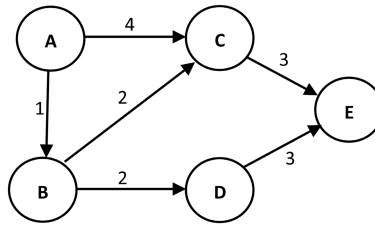


Figure 8.23 Dijkstra's algorithm for the graph

Step 1: Let A be the source vertex.

```

 $d(A) = 0$ ;
 $d(B) = d(C) = d(D) = d(E) = \infty$ 

```

Step 2: Find adjacent vertices of A.

```

Adj(A) = {B, C};
 $d(B) = 4$  ;  $d(C) = 1$  since they are less than  $\infty$ .
Pred(B) = A and pred(C) = A;

```

Step 3: Choose the vertex with a smaller distance value, which is C.

```

Adj(C) = {B, D}.
Update  $d(B)$  and  $d(D)$ 
 $d(B) > d(C) + w(C,B)$  //  $w(C,B)$  is the weight on the edge C->B
Therefore,  $d(B) = d(C) + w(C,B) = 1 + 2 = 3$ ;
Pred(B) = C;
 $d(D) = d(C) + w(C,D) = 1 + 2 = 3$ ;
pred(D) = C;

```

Step 4

```

Adj(B) = {E};
 $d(E) = d(B) + w(B,E) = 3 + 3 = 6$ ;
pred(E) = B;

```

Step 5

$$\begin{aligned}\text{Adj}(D) &= E; \\ d(E) &= d(D) + w(D, E) = 3 + 3 = 6\end{aligned}$$

which is the same value computed earlier and we keep the earlier value and earlier predecessor. We show the shortest paths in the following diagram.

Step 6

$$\text{Adj}(E) = \varnothing.$$

This means there are no outgoing edges from E. And there are no more vertices. The algorithm terminates.

Using the information about the predecessors of each vertex, we show the shortest paths to each vertex from A (Figure 8.24).

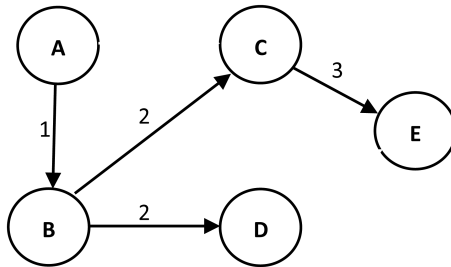


Figure 8.24 Shortest path after Dijkstra's algorithm

8.7 TOPOLOGICAL SORTING

Definition: A directed graph G in which vertices represent tasks or activities and edges represent precedence relations between tasks, is called an activity on vertex network (AOV). Topological sort is applied to an activity network to determine the order in which tasks are to be executed.

Algorithm 3

1. Algorithm topological sort.
2. Input the AOV network and let n be the number of vertices.
3. for $i = 1$ to n do
4. {
- 4.1. If every vertex has a predecessor, then the network has a cycle and is infeasible. Stop
5. Pick a vertex v that has no predecessors.
6. Output v .
7. Delete v and all edges leading out of v .
8. }

Example: Consider the activity network in Figure 8.25a.

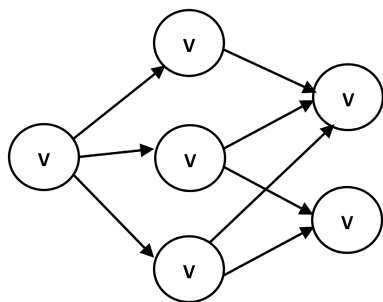


Figure 8.25a Topological sorting for the graph

Since v_1 has no predecessors, remove v_1 and all edges leading out of it (Figure 8.25b).

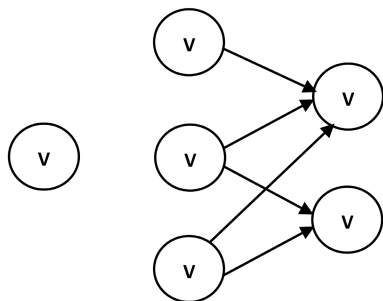


Figure 8.25b Topological sorting after removing edges from v_1

Now neither v_2 , nor v_3 , nor v_4 have any predecessors. Choose any one of them, say, v_2 and remove v_2 and all edges leading out of it (Figure 8.26).

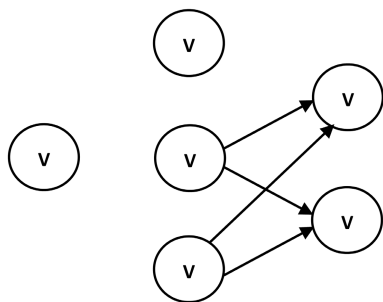


Figure 8.26 Topological sorting after removing edges from v_2

Now remove v_3 and all edges leading out of it (Figure 8.27).

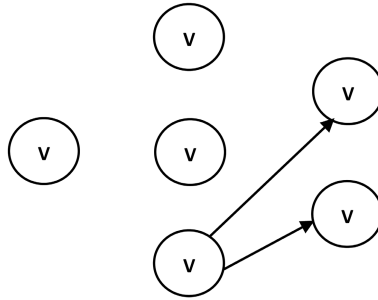


Figure 8.27 Topological sorting after removing edges from v_3

We get $v_1 v_2 v_3 v_4 v_5 v_6$ as the order in which the activities are to be executed.

True–false questions

1. A graph is a tree.
 - a. True
 - b. False
2. An edge is a connection between two vertices.
 - a. True
 - b. False
3. A bi-connected graph is not essentially a connected graph.
 - a. True
 - b. False
4. The shortest path between two vertices in the graph is the least number of edges.
 - a. True
 - b. False
5. A queue is used for depth-first search.
 - a. True
 - b. False

Fill-in-the-blank questions

1. A graph can be defined (and represented) as a set of _____ and _____.
2. Vertices in _____ are stored in an arbitrary order.
3. The graph in which all the nodes have the same degree is called a _____.
4. When labels are associated with the edges, a graph with numbers indicating the importance of the edge is called a _____.
5. A _____ in a graph is a directed path from any vertex to itself.

Multiple-choice questions

1. An AND/OR graph is a
 - a. Multiway tree
 - b. Binary tree
 - c. Not necessarily a tree
 - d. Not necessarily a binary tree

2. The total number of edges associated with a node is called
 - a. In-degree
 - b. Out-degree
 - c. Degree
 - d. None of the above

3. Which of the following statements is/are true for undirected graphs?
P: number of odd degree vertices is even.
Q: sum of all vertices is even.
 - a. P only
 - b. Q only
 - c. Both P and Q
 - d. None of the above

4. A tree with n vertices has
 - a. n edges
 - b. $N-1$ edges
 - c. $n+1$ edges
 - d. A cycle in the graph

5. A graph with one vertex and no edges is a/an
 - a. Isolated graph
 - b. Multigraph
 - c. Trivial graph
 - d. Di-graph

6. A graph in which all nodes are of equal degree is known as a
 - a. Di-graph
 - b. Complete graph
 - c. Multigraph
 - d. Regular graph

7. A spanning tree of a graph is one that includes
 - a. All edges of the graph
 - b. All vertices of the graph
 - c. Only vertices with even degrees
 - d. None of the above

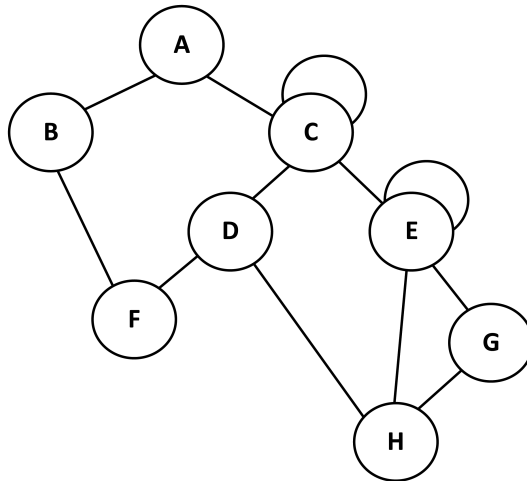
8. The degree of any vertex of a graph is
 - a. The number of edges incident with the vertex
 - b. Number of vertices in the graph
 - c. Number of vertices adjacent to that vertex
 - d. Number of edges in the graph

9. A simple graph that contains exactly one edge between each pair of distinct vertices is a
 - a. Tree
 - b. Degree
 - c. Path
 - d. Complete graph
10. When information about the vertices is more desirable than information about the edges, we use
 - a. Adjacency matrix
 - b. Adjacency list
 - c. Both a and b
 - d. None of the above
11. In an adjacency matrix, if any diagonal element is 1, it indicates
 - a. There is a self-loop.
 - b. There is a path.
 - c. There is a cycle.
 - d. None of the above.
12. What is a sequence of edges that begins at a vertex of a graph and travels along edges of the graph, always connecting pairs of adjacent vertices?
 - a. Leaf nodes
 - b. Internal nodes
 - c. Path
 - d. Cycle
13. What is the removal of a vertex known as when it produces a subgraph with more connected components than in the original graph?
 - a. Connected graph
 - b. Articulation point
 - c. Complete Graph
 - d. Even degree
14. What is the maximum number of edges in a acyclic undirected connected graph with n vertices?
 - a. $n/2$
 - b. $n-1$
 - c. n
 - d. $n+1$
15. What is the maximum degree of any node in a simple graph with n vertices?
 - a. $n/2$
 - b. $n-1$
 - c. n
 - d. $n+1$
16. The number of circuits in a tree is
 - a. 0
 - b. 1
 - c. 2
 - d. 3

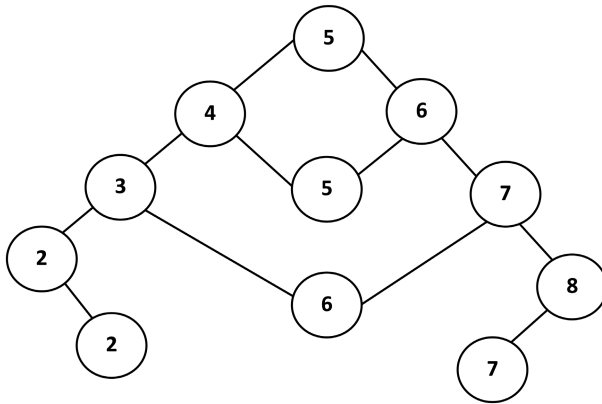
17. The degree of each vertex in K_n is
- | | |
|-----------|----------|
| a. $2n-1$ | c. $n-1$ |
| b. n | d. $N-2$ |
18. What is the number of distinct simple graphs with up to three nodes?
- | | |
|------|-------|
| a. 7 | c. 10 |
| b. 9 | d. 15 |
19. What is the number of vertices in an undirected connected graph with 27 edges, 6 vertices of degree 2, 3 vertices of degree 4 and the remaining of degree 3?
- | | |
|-------|-------|
| a. 10 | c. 18 |
| b. 11 | d. 19 |
20. In any undirected graph, the sum of degrees of all the nodes
- | | |
|------------------------------|----------------------|
| a. Must be even | c. Both a and b |
| b. Twice the number of edges | d. None of the above |

Descriptive questions

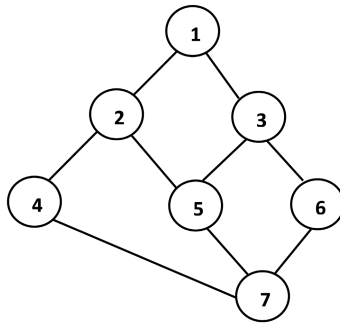
1. Find the degree of each node in the graph given below.



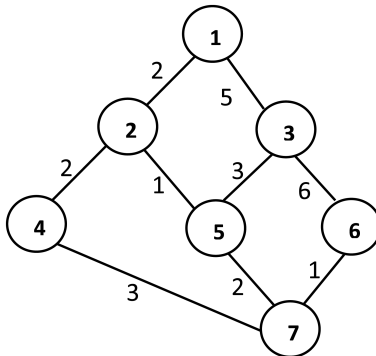
2. Find the bi-connected components of the graph given below.



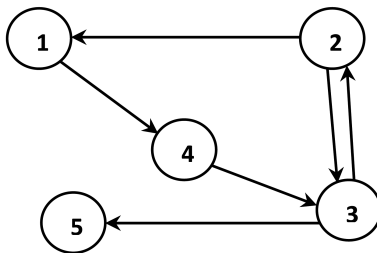
3. Find the order of nodes that is printed in DFS for the following graph.



4. Find the adjacency matrix and adjacency lists for the following graph.



- 5. What is an AND/OR graph and how it is useful in problem-solving?
- 6. Show that a circuitless connected graph with n vertices has $n-1$ edges.
- 7. Use the topological sort for the following graph.



- 8. Explain different types of graph traversal techniques with examples.
- 9. What is a directed acyclic graph and what is the usefulness of it?
- 10. Explain the following with examples.
 - i. Weighted graph
 - ii. Edge-labeled graph
 - iii. Cyclic graph

Answers to true–false questions

- 1. False
- 2. True
- 3. False
- 4. True
- 5. False

Answers to fill-in-the-blank questions

- 1. vertices, edges
- 2. adjacency lists
- 3. complete graph
- 4. weighted graph
- 5. cycle

Answers to multiple-choice questions

1.	c	2.	c	3.	c	4.	b	5.	c
6.	d	7.	b	8.	a	9.	d	10.	c
11.	a	12.	c	13.	b	14.	b	15.	b
16.	a	17.	c	18.	a	19.	b	20.	c